COLLEGE FINAL VIVA

SHOPZY

Backend Technical Viva & Defense Master Guide

Hyperlocal Business Discovery & Real-Time Price Comparison Engine

Project Viva Snapshot & Technical Stack:

* Runtime & Language : Node.js (v20+), Express.js 4.21, TypeScript 5.7
* Database Engine : MongoDB 8.0+ with 2dsphere Geospatial Indexing & GeoJSON Points
* Caching Layer : Redis (ioredis) with Cache-Aside Strategy & Fallback Circuit
* Security & RBAC : Dual-Token JWT (Access + Refresh), Bcrypt (10 rounds), Helmet, HPP, CSRF
* Schema Validation : Zod 3.24 (Type-safe runtime validator & TypeScript inference)
* File Handling : Multer (UUID renaming, MIME-type whitelisting, 5MB caps)
* Quality Assurance : Jest 29 & Supertest (150 passing unit and integration tests)

KEY STRATEGY FOR TOMORROW'S PRESENTATION / VIVA:

1. Examiners ask 'WHY' more than 'WHAT' - Always explain why MongoDB was chosen over SQL, why Redis was added, and why dual-token JWT is safer than simple sessions.
2. Highlight Geospatial 2dsphere indexing and Price Comparison aggregations - These are your project's high-tech core.
3. Mention Error Handling and Race Condition mitigation (atomic $inc and $addToSet) to show senior engineering depth.
4. Each question below has a 'Hindi Tip' so you understand the intuition, plus crisp technical English for your viva answer.

Index of Sections Covered:

Section 1: System Architecture, Design Decisions & Monorepo Structure (Q1 - Q5)
Section 2: Database Schema Design & 2dsphere Geospatial Queries (Q6 - Q10)
Section 3: Authentication, Dual-Token Architecture & Security Hardening (Q11 - Q15)
Section 4: Performance Caching (Redis), Resiliency & Optimization (Q16 - Q20)
Section 5: Concurrency, Race Conditions & Data Integrity (Q21 - Q24)
Section 6: API Architecture, Zod Validation Pipeline & Error Handling (Q25 - Q28)
Section 7: Complex Business Logic & Feature Implementations (Q29 - Q32)
Section 8: Testing Strategy (Jest/150 Tests), DevOps & Viva Success Tips (Q33 - Q36)

SECTION 1: SYSTEM ARCHITECTURE & DESIGN DECISIONS

Questions on overall structure, layering, TypeScript usage, and architectural trade-offs.

Q1. Explain the overall backend architecture of Shopzy. Why Layered Architecture?

Examiner Focus: Examiner checks whether you know separation of concerns or just dumped everything in index.js. | Hindi Tip: Sir ko batao: Routes sirf URL map karte hain, Controllers HTTP request/response handle karte hain, Services business logic run karte hain, aur Models DB se baat karte hain.

Shopzy follows a 4-Tier Layered Architecture:

1. Routing Layer (server/src/routes): Maps REST endpoints and attaches security/validation middleware.
2. Controller Layer (server/src/controllers): Parses req.params/query/body, delegates to services, and shapes the HTTP response.
3. Service Layer (server/src/services): Contains pure, reusable business logic (e.g. searchService, auditService, shopService).
4. Data Access Layer (server/src/models): Mongoose schemas, indexes, and aggregation pipelines.

Why Layered Architecture? It decouples business logic from HTTP transport. Services can be independently unit-tested without mocking Express Request/Response objects, and if we ever migrate from Express to Fastify or NestJS, 90% of business logic remains untouched.

```
Data Flow: Client HTTP Request -> Helmet/CORS -> RateLimiter -> Router -> Auth Middleware -> Validator(Zod) -> Controller -> Service -> Model(MongoDB) -> Response Helper
```

Q2. Why did you choose Node.js with TypeScript instead of Python (Django/FastAPI) or Java (Spring Boot)?

Examiner Focus: Examiner evaluates whether your tech stack choice was intentional or just random. | Hindi Tip: Node.js asynchronous event loop I/O heavy hyperlocal searches ke liye perfect hai. TypeScript runtime type errors ko compile time par hi rok deta hai.

1. Asynchronous Non-Blocking I/O: Shopzy is an I/O-heavy platform (fetching geo-searches, caching in Redis, logging analytics). Node.js single-threaded event loop handles thousands of concurrent I/O requests with minimal memory footprint compared to thread-per-request models (Spring Boot).
2. Single Language Across Full Stack: Both web client (React 19) and backend (Express) share TypeScript types and Zod schemas, eliminating interface mismatches.
3. TypeScript Safety: Compile-time typechecking prevents common production crashes like 'Cannot read property of undefined' and enforces strict API contracts across controllers and services.

```
Key Metric: Shopzy's server boots in ~300ms, consumes <90MB RAM idle, and achieves sub-50ms query response times.
```

Q3. Why a Modular Monolith with Workspaces instead of Microservices?

Examiner Focus: Classic viva question to test if you over-engineered for a college project. | Hindi Tip: Sir ko bolo: Early stage me Microservices me network latency, distributed transactions, aur DevOps overhead hota hai. Humne Modular Monolith chuna jo scalable hai aur complex network failure avoid karta hai.

We chose a Modular Monolith configured as an npm workspace ('server' and 'web').

Reasons:

1. Operational Simplicity: Microservices introduce distributed transaction overhead (Saga pattern), network latency between services, and complex service mesh management (Kubernetes/Istio).
2. Clean Domain Boundaries: Our codebase is strictly decoupled by domain (Auth, Shops, Reviews, Categories, Analytics, Audit). If scaling requires it, any domain (like Search/Geo engine) can be sliced into an independent microservice with zero rewrites.
3. Shared Tooling & Testing: A single repository enables unified linting, CI/CD, and fast integration testing with supertest in milliseconds.

```
Examiner Punchline: 'Premature decomposition into microservices is the root of many startup failures.
```

```
A clean modular monolith is the industry best practice until team size exceeds 20+ developers.'
```

Q4. What Design Patterns have you implemented in the backend?

Examiner Focus: Examiner wants to hear standard Gang of Four (GoF) or enterprise patterns. | Hindi Tip: Middleware pattern, Singleton for database/cache, Factory pattern for responses, aur Cache-Aside pattern.

1. Middleware Pattern (Chain of Responsibility): Used across Express pipeline for auth, validation, rate limiting, and global error handling.
2. Singleton Pattern: MongoDB connection pool and Redis client (utils/cache.ts) are initialized once as singletons across the entire process lifetime.
3. Cache-Aside Pattern: In utils/cache.ts, cached() checks Redis first; on cache miss, executes the DB supplier, stores result with TTL, and returns.
4. Standard Response Factory: utils/response.ts provides ok(), created(), and buildPageMeta() to enforce uniform JSON structure across all endpoints.
5. Higher-Order Function Pattern: asyncHandler wraps every async controller function to forward errors to the global error handler automatically.

```
Example Code Reference: server/src/utils/asyncHandler.ts & server/src/utils/cache.ts
```

Q5. How is CORS and Security Headers configured? Why is it crucial?

Examiner Focus: Checks understanding of web security basics. | Hindi Tip: CORS allow karta hai sirf hamare frontend (localhost:5173 ya production domain) ko API access karne ke liye. Helmet 11+ security HTTP headers set karta hai.

1. CORS (Cross-Origin Resource Sharing): Configured via cors() middleware with explicit whitelist (origin, methods: GET, POST, PUT, PATCH, DELETE, credentials: true for cookies). Malicious third-party websites cannot initiate authenticated API calls from an unauthorized browser.
2. Helmet: Injects security headers like Content-Security-Policy, X-Frame-Options (prevents Clickjacking), X-Content-Type-Options: nosniff (prevents MIME sniffing), and Strict-Transport-Security (HSTS).
3. HPP (HTTP Parameter Pollution): hpp() protects against array parameter injection attacks (e.g. ?role=customer&role=admin).

```
Code Location: server/src/index.ts -> app.use(helmet()), app.use(cors(corsOptions)), app.use(hpp())
```

SECTION 2: DATABASE SCHEMA & GEOSPATIAL INDEXING

The technical core of Shopzy: GeoJSON, 2dsphere indexing, spatial queries, and price comparison pipelines.

Q6. Why MongoDB (NoSQL) over relational databases (PostgreSQL/MySQL) for this project?

Examiner Focus: Examiner tests whether you understand document vs relational data models. | Hindi Tip: Shops ka data flexible aur polymorphic hota hai (kisi ke paas menu items hain, kisi ke paas services, timing alag hai). Plus GeoJSON aur 2dsphere MongoDB me natively optimized hain.

1. Polymorphic Shop Catalog: A restaurant has menu items with calories and dietary tags; a salon has haircut packages; a gym has monthly/annual memberships. In SQL, this requires 5+ joined tables with complex foreign keys. MongoDB embeds items directly inside pricing.items with dynamic attributes.
2. Native GeoJSON Support: MongoDB provides first-class support for Point coordinates [longitude, latitude] with spherical distance calculations built directly into the C++ query engine.
3. Rapid Aggregation: Aggregation pipelines allow multi-stage facet filtering, keyword matching, and geo-sorting in a single round-trip without heavy table locks.

```
Schema Reference: server/src/models/Shop.ts (embeds location, operatingHours, pricing.items, offers)
```

Q7. How does Hyperlocal / Proximity Search work? Explain the 2dsphere index.

Examiner Focus: MOST IMPORTANT QUESTION OF YOUR ENTIRE VIVA. 100% guarantee examiner will ask this. | Hindi Tip: Shop model me location.coordinates par 2dsphere index lagaya hai. Format hamesha [longitude, latitude] hota hai. MongoDB Earth ki curvature consider karke exact distance calculate karta hai.

1. GeoJSON Point Structure: Shop coordinates are stored as:
location: { type: 'Point', coordinates: [longitude, latitude] }
*CRITICAL: Longitude comes FIRST, then Latitude (X, Y order as per GeoJSON RFC 7946).

2. 2dsphere Index: Created using shopSchema.index({ 'location.coordinates': '2dsphere' }). Unlike a flat 2d index (Euclidean plane), 2dsphere calculates distances over an Earth-like WGS84 sphere, taking curvature into account.

3. Query Execution: When a user searches near Bhopal Upper Lake (lat: 23.25, lng: 77.40, radius: 5km):

```
Shop.find({
  'location.coordinates': {
    $near: { $geometry: { type: 'Point', coordinates: [77.40, 23.25] }, $maxDistance: 5000 }
  },
  isActive: true
})
```

MongoDB uses spatial B-tree index to return shops sorted nearest-first in <15ms without scanning all rows.

```
Mongoose Definition: ShopSchema.index({ 'location.coordinates': '2dsphere', 'basicInfo.category': 1 })
```

Q8. What is the difference between $near and $geoNear? Which did you use and why?

Examiner Focus: Distinguishes an average student from a top-grade student. | Hindi Tip: $near simple query me chalta hai lekin output me distance calculate karke field return nahi karta. $geoNear aggregation pipeline ka first stage hota hai aur har shop ka exact distanceInKm return karta hai.

1. $near:
- Used in basic Shop.find() queries.
- Sorts results by distance automatically, but DOES NOT add a calculated 'distance' field to the returned JSON document.

2. $geoNear:
- Must be the FIRST stage in an Aggregation Pipeline ($geoNear).
- Computes exact distance in meters and projects it as a new calculated field (e.g. distanceMeters / distanceKm).

- Allows downstream stages like $match (rating >= 4), $facet, and $project.

In Shopzy: We use $geoNear in searchService.searchShops() and comparePrices() because the frontend needs to show '1.2 km away' on the shop card!

```
Aggregation Stage: { $geoNear: { near: { type: 'Point', coordinates: [lng, lat] }, distanceField: 'distance', maxDistance: radius * 1000, spherical: true } }
```

Q9. Explain the Price Comparison Aggregation Pipeline. How does it compare prices across shops?

Examiner Focus: Crucial USP question: How does Shopzy compare item prices side-by-side? | Hindi Tip: Aggregation pipeline use hoti hai: Pehle $geoNear se nearby shops nikalte hain, phir $unwind se pricing.items array kholte hain, item name par regex match karte hain, aur cheapest item pehle sort karte hain.

The price comparison engine (server/src/services/search.service.ts -> comparePrices) runs a 5-stage MongoDB pipeline:
Stage 1: $geoNear - Filters active shops within radius (e.g. 10km) and calculates distance.
Stage 2: $unwind: '$pricing.items' - Flattens the embedded price list array into individual item documents.
Stage 3: $match - Filters items matching the searched query using case-insensitive regex (e.g. /haircut/i or /cappuccino/i) and isAvailable: true.
Stage 4: $project - Shapes clean comparison row: shopId, shopName, itemName, price, comparePrice, savingsPercentage, distance.
Stage 5: $sort - Sorts by item.price: 1 (cheapest first), then distance: 1.
Also computes summary stats: minPrice, maxPrice, avgPrice across all nearby shops.

```
Code File: server/src/services/search.service.ts (Function: comparePrices)
```

Q10. How do you handle Database Indexing and Performance in MongoDB?

Examiner Focus: Examiner checks if your database will slow down when data grows. | Hindi Tip: Compound indexes lagaye hain jaise category + verification status, text index lagaya hai shop name aur tags par, aur explain('executionStats') se query plan verify kiya hai.

1. Compound Indexes: For frequent multi-key queries:
   - { 'basicInfo.category': 1, 'verification.status': 1, isActive: 1 }
   - { userId: 1, shopId: 1 } on Reviews (ensures 1 review per user per shop).
2. Text Index: { 'basicInfo.name': 'text', 'basicInfo.tags': 'text', 'basicInfo.description': 'text' } for full-text fuzzy keyword matching.
3. Lean Queries: Using .lean() in Mongoose skips hydrating full Mongoose documents into memory, saving 60% memory overhead and speeding up JSON serialization by 3x.
4. Projection: Explicitly selecting required fields (.select('basicInfo.name location.address ratings')) avoids fetching heavy image arrays unnecessarily.

```
Reference: Mongoose models in server/src/models/Shop.ts and server/src/models/Review.ts
```

SECTION 3: AUTHENTICATION, RBAC & SECURITY HARDENING

JWT dual-token architecture, password hashing, role gates, and protection against common attacks.

Q11. How is Authentication implemented? Explain the Dual-Token Strategy.

Examiner Focus: Core backend security question. | Hindi Tip: Sir, single token vulnerable hota hai. Humne Dual-Token banaya: Access Token (short-lived, 15m) memory me rehta hai, aur Refresh Token (long-lived, 7d) httpOnly cookie me rehta hai.

Shopzy implements the industry-standard Dual-Token JWT Architecture:

1. Access Token (Short-lived: 15 minutes):
   - Encoded with userId, email, and role.
   - Sent in the Authorization: Bearer <token> header.
   - Used to verify every API request statelessly without querying database.

2. Refresh Token (Long-lived: 7 days):
   - Stored in an httpOnly, SameSite=Lax, Secure cookie (inaccessible to JavaScript, immune to XSS theft).
   - Stored as an encrypted hash in the database User document.

3. Silent Refresh Cycle: When Access Token expires (401 response), the frontend Axios/Fetch interceptor automatically calls /api/v1/auth/refresh with the cookie. The server validates the refresh token, verifies user is active, and issues a fresh Access Token seamlessly without forcing user to re-login.

```
Code Reference: server/src/controllers/auth.controller.ts & server/src/middleware/auth.ts
```

Q12. How are passwords stored securely? Why Bcrypt over SHA-256 or MD5?

Examiner Focus: Examiner checks cryptography knowledge. | Hindi Tip: SHA-256 fast hota hai jisse attackers GPU se rainbow tables use karke crack kar sakte hain. Bcrypt adaptive work factor (salt rounds) use karta hai jo brute-force ko computationally impossible banata hai.

1. Why NOT MD5 / SHA-256: MD5 and SHA-256 are cryptographic hash functions designed to be FAST. Modern GPUs can compute over 10 billion SHA-256 hashes per second, making brute-force and precomputed Rainbow Table attacks trivial.
2. Why Bcrypt: Bcrypt is an adaptive key-derivation function based on the Blowfish cipher. It includes:
   - Automatic Random Salt (10 rounds): Salt is unique per user, preventing rainbow tables.
   - Cost Factor (Work Factor): Intentionally computationally expensive (~80ms per hash), making offline brute-force attacks infeasible.
3. Implementation: Mongoose userSchema.pre('save') hook automatically hashes password before persisting if modified, and user.comparePassword() runs bcrypt.compare().

```
Implementation: server/src/models/User.ts -> bcrypt.hash(this.password, 10)
```

Q13. How does Role-Based Access Control (RBAC) work? How do you prevent Horizontal Privilege Escalation?

Examiner Focus: Security & Authorization test. | Hindi Tip: RBAC me 4 roles hain: customer, owner, admin, super_admin. Horizontal privilege escalation rokne ke liye controller check karta hai ki kya req.user.id shop.ownerId ke barabar hai.

1. Role Verification Middleware (requireRole):
   - Placed on routes after authenticate middleware.
   - Checks if allowedRoles.includes(req.user.role). If not, immediately throws ApiError.forbidden('Insufficient permissions').

2. Preventing Horizontal Privilege Escalation (Insecure Direct Object Reference - IDOR):
   - Problem: What if Shop Owner A tries to edit Shop B's prices by sending PUT /api/v1/shops/:shopB_id?
   - Solution: In shop.controller.ts (updateShop):
     const shop = await Shop.findById(shopId);
     const isOwner = shop.ownerId.toString() === req.user.id;

const isAdmin = ['admin', 'super_admin'].includes(req.user.role);
if (!isOwner && !isAdmin) throw ApiError.forbidden('You do not own this shop');
- Even if an attacker guesses another shop's ID, the server rejects unauthorized mutations with 403 Forbidden.

```
Code Reference: server/src/middleware/auth.ts (requireRole) & server/src/controllers/shop.controller.ts
```

Q14. How does your backend defend against OWASP Top 10 vulnerabilities?

Examiner Focus: Examiner checks real-world production security readiness. | Hindi Tip: SQL/NoSQL injection Zod aur Mongoose types se rokte hain, XSS ko httpOnly cookie aur Helmet headers se, CSRF ko double submit se, aur DDoS ko express-rate-limit se.

1. NoSQL Injection: Attackers sending { username: { $gt: '' } } are stopped because Zod schemas strictly parse body to string primitives before queries run.
2. Cross-Site Scripting (XSS): Tokens are stored in httpOnly cookies so document.cookie cannot read them. Content-Security-Policy header blocks inline script execution.
3. Cross-Site Request Forgery (CSRF): Sensitive state-changing requests validate CSRF double-submit cookies or custom X-CSRF-Token headers.
4. DDoS & Brute-Force: express-rate-limit caps requests at 100 requests per 15-min window per IP, and /auth/login is throttled strictly to 10 attempts.
5. Broken Object Level Authorization (BOLA/IDOR): Owner IDs verified on every single document mutation.

```
Reference: server/src/middleware/rateLimiter.ts & server/src/middleware/csrf.ts
```

Q15. What happens when a user logs out or is suspended by Admin?

Examiner Focus: Session revocation and token invalidation. | Hindi Tip: Logout par client-side cookie clear hoti hai aur DB me user ka refresh token null set ho jata hai. Admin suspend kare to isActive: false hota hai jisse next request par token reject ho jata hai.

1. Logout: /api/v1/auth/logout sets refreshToken = null on the User document in MongoDB and clears the HTTP cookie with res.clearCookie('refreshToken').
2. Admin Suspension: When an admin suspends a rogue user (PATCH /api/v1/admin/users/:id/status -> status: 'suspended', isActive: false):
   - The authenticate middleware checks User.findById(decoded.id).select('isActive status').
   - If isActive === false, it immediately throws ApiError.unauthorized('Account suspended').
   - Even if the attacker holds an unexpired 15-minute access token, their access is terminated immediately on the next API call.

```
Code Location: server/src/controllers/auth.controller.ts -> logout & server/src/middleware/auth.ts
```

SECTION 4: PERFORMANCE, REDIS CACHING & RESILIENCY

Cache-aside architecture, TTL invalidation, fallback mechanisms, and high-concurrency optimizations.

Q16. Why did you use Redis? What data is cached and what is the invalidation strategy?

Examiner Focus: High-performance architecture question. | Hindi Tip: Redis in-memory cache hai jo disk I/O bachata hai. All Categories aur Trending Searches ko Redis me cache kiya hai. Jab Admin category update kare to cache turant delete ho jata hai.

1. Why Redis: MongoDB disk-reads take 15-40ms. Redis stores data in RAM with O(1) key lookups, serving cached responses in <1ms.

2. What is Cached:
- categories:all (CacheTTL: 1 hour / 3600s) - Read by every visitor on the homepage.
- searches:trending (CacheTTL: 15 mins / 900s) - Trending search queries.
- suggestions:autocomplete:<q> (CacheTTL: 5 mins / 300s) - Nav search debounced queries.

3. Cache Invalidation Strategy:
- We use the Cache-Aside pattern with Event-Driven Invalidation.
- Whenever an Admin creates, updates, or deletes a Category (server/src/controllers/category.controller.ts):
  await cache.del('categories:all');
- This purges stale cache immediately, ensuring the very next user query fetches fresh DB data and repopulates Redis.

```
Implementation: server/src/utils/cache.ts (Functions: cached(), cache.get(), cache.set(), cache.del())
```

Q17. What happens if Redis crashes? Does the entire Shopzy platform crash? (Resiliency)

Examiner Focus: Examiner checks system fault tolerance and graceful degradation. | Hindi Tip: Nahi sir! Humne try/catch circuit breaker wrapper banaya hai. Agar Redis down ho jaye to system crash nahi hota, seedhe MongoDB se data leke serve karta hai.

No, the platform does NOT crash. Shopzy implements Graceful Degradation / Circuit Fallback:
1. In server/src/utils/cache.ts, the cached() wrapper catches Redis connection errors:

```
try {
  const cachedVal = await redis.get(key);
  if (cachedVal) return JSON.parse(cachedVal);
} catch (err) {
  logger.warn(`Redis unavailable, falling back to DB: ${err.message}`);
}
// Fallback supplier executed directly against MongoDB
const freshData = await supplier();
return freshData;
```

2. Optional Dependency: ioredis is configured as an optional dependency so local development can run even without Redis installed.
3. This is verified by our automated regression tests (tests/regressions.test.ts) where tests pass even when Redis is offline.

```
Test Evidence: tests/regressions.test.ts passes with warning 'Rate limiter unavailable, allowing request: redis is down'
```

Q18. How does HTTP Compression and Payload Optimization work?

Examiner Focus: Bandwidth and mobile performance optimization. | Hindi Tip: compression() middleware use kiya hai jo responses ko Gzip/Deflate karke send karta hai, jisse JSON payload size 70% chhota ho jata hai.

1. Gzip Compression: Express compression() middleware dynamically compresses response bodies exceeding 1KB using Gzip/Brotli.
2. Impact: A search result payload with 20 shops and embedded pricing items shrinks from ~85KB to ~14KB (an 83% bandwidth reduction), significantly improving mobile load times in low-network regions.
3. Cache-Control & ETag: Express generates weak ETags for static JSON responses. If frontend requests with If-None-Match matching the hash, server returns 304 Not Modified without sending the response body.

```
Code Location: server/src/index.ts -> app.use(compression())
```

Q19. Explain MongoDB Connection Pooling in Mongoose.

Examiner Focus: Database connection management. | Hindi Tip: Har API request par naya DB connection kholne se server crash ho jayega. Connection pool 50 pre-warmed connections maintain karta hai jo reuse hote hain.

1. The Problem: Opening a TCP socket and authenticating a TLS handshake with MongoDB takes 50-100ms per request. Creating a new connection per request would quickly exhaust DB file descriptors and crash under traffic.
2. Connection Pooling: Mongoose manages an internal pool of persistent, pre-authenticated sockets:

```
mongoose.connect(MONGO_URI, {
  maxPoolSize: 50,         // Up to 50 concurrent DB operations
  minPoolSize: 10,         // Always keep 10 connections warm
  serverSelectionTimeoutMS: 5000,
  socketTimeoutMS: 45000
});
```

3. Requests borrow an idle connection from the pool, execute their query in <5ms, and return it to the pool immediately.

```
Code Location: server/src/config/db.ts
```

SECTION 5: CONCURRENCY, RACE CONDITIONS & DATA INTEGRITY

Handling simultaneous updates, atomic operators, rating aggregations, and cascading deletions.

Q20. What happens if 100 users bookmark/save a shop at the exact same millisecond? (Race Conditions)

Examiner Focus: Examiner tests knowledge of atomic database operations vs read-modify-write. | Hindi Tip: Agar read-modify-write karenge to counter overwrite ho jayega. Humne MongoDB atomic operators $addToSet, $pull, aur $inc use kiya hai jo server-side atomic lock lagata hai.

1. The Naive Flaw (Read-Modify-Write):

```
const user = await User.findById(id);
user.savedShops.push(shopId);
await user.save(); // RACE CONDITION! Simultaneous saves overwrite each other.
```

2. The Shopzy Atomic Solution (user.controller.ts -> toggleSavedShop):

We use MongoDB atomic operators $addToSet and $inc:

```
const before = await User.findOneAndUpdate(
  { _id: userId },
  { $addToSet: { savedShops: objectId } },
  { new: false, projection: { savedShops: 1 } }
);
const already = before.savedShops.some(id => id.equals(objectId));
if (already) {
  await User.updateOne({ _id: userId }, { $pull: { savedShops: objectId } });
}
// Only increment/decrement counter if state actually changed
await Shop.updateOne({ _id: shopId }, { $inc: { 'analytics.saveCount': already ? -1 : 1 } });
```

Result: Even under 1,000 parallel requests, MongoDB serializes atomic document locks. No duplicate bookmarks can ever exist, and saveCount never drifts.

`Code Reference: server/src/controllers/user.controller.ts (Line 35-52)`

Q21. How do you calculate and maintain a Shop's Average Rating and Review Count?

Examiner Focus: Real-time calculations vs background aggregation. | Hindi Tip: Jab user review add ya delete karta hai, hum Review collection par aggregate pipeline run karke $avg aur count nikalte hain aur Shop document me update karte hain.

1. Why NOT recalculate on every shop visit: If a popular shop has 5,000 reviews, calculating average rating on every pageview would require reading 5,000 review documents, destroying response times.

2. Post-Save Aggregation Trigger: Whenever a review is created, edited, or deleted (server/src/models/Review.ts static method recalculateShopRating):

```
const stats = await Review.aggregate([
  { $match: { shopId: new mongoose.Types.ObjectId(shopId), isActive: true } },
  { $group: {
      _id: '$shopId',
      count: { $sum: 1 },
      avgRating: { $avg: '$rating' },
      breakdown: {
        $push: '$rating'
      }
  }}
]);
```

```
  await Shop.findByIdAndUpdate(shopId, {
    'ratings.average': Math.round(stats[0].avgRating * 10) / 10,
    'ratings.count': stats[0].count
  });
```

3. Result: Shop detail and search queries read precomputed ratings.average directly in O(1) time.

`Code Reference: server/src/models/Review.ts & server/src/controllers/review.controller.ts`

Q22. How do you handle Cascading Deletions / Orphaned Records?

Examiner Focus: Database integrity without SQL foreign key CASCADE constraints. | Hindi Tip: MongoDB me foreign keys CASCADE nahi hota. Isliye jab admin shop delete karta hai to controller shop ke reviews, saved references, aur uploaded images ko bhi cleanup karta hai.

In NoSQL databases, there is no native ON DELETE CASCADE foreign key constraint. Shopzy implements Application-Level Cascading Deletion in admin.controller.ts (deleteShop):

1. Review Deletion: When a shop is deleted, all associated reviews are soft-deleted or removed: await Review.deleteMany({ shopId });

2. Customer Saved Lists: Removes shopId from all customer bookmarks: await User.updateMany({}, { $pull: { savedShops: shopId } });

3. Physical Image Deletion: Iterates through shop.images array and unlinks physical files from disk (/uploads/) to prevent disk memory leaks.

4. Audit Logging: Logs the deletion event in the immutable Audit collection with the admin's IP, timestamp, and shop snapshot.

`Code Location: server/src/controllers/admin.controller.ts (Function: deleteShop)`

SECTION 6: API PIPELINE, ZOD VALIDATION & ERROR ARCHITECTURE

Request validation, custom ApiError hierarchy, asyncHandler, and standard REST responses.

Q23. Why Zod for Schema Validation? How does it differ from Joi or express-validator?

Examiner Focus: Modern validation pipeline question. | Hindi Tip: Zod TypeScript-first validator hai. z.infer se ek hi schema se runtime validation aur static TypeScript types dono mil jaate hain. Double coding nahi karni padti.

1. TypeScript Type Inference: With Joi or express-validator, you have to write the Joi schema AND manually write an interface CreateShopDTO. If you change a field, you must update both, leading to bugs. With Zod:

```
export const createShopSchema = z.object({
  name: z.string().min(2).max(100),
  phone: z.string().regex(/^[0-9]{10}$/),
  category: z.string()
});
export type CreateShopInput = z.infer<typeof createShopSchema>;
```

2. Strict Runtime Validation: If the client sends unexpected extra fields or incorrect types, Zod rejects the request with HTTP 400 Bad Request and detailed field-level error messages before controller logic runs.

3. Code Cleaner: Our validate() middleware wraps any Zod schema: router.post('/', validate(createShopSchema), controller).

`Code Reference: server/src/middleware/validate.ts & server/src/schemas/`

Q24. How do you handle Asynchronous Errors in Express 4 without crashing the Node.js process?

Examiner Focus: Critical Express.js runtime question. | Hindi Tip: Express 4 async functions ke unhandled promise rejections catch nahi karta. Humne asyncHandler wrapper banaya jo Promise.resolve().catch(next) karta hai.

1. The Problem in Express 4: If an async route handler throws an error (e.g. await Shop.create() fails) and is not wrapped in a try/catch, Express 4 DOES NOT pass it to the error-handling middleware. It triggers an unhandledRejection event, which in Node.js 16+ crashes the entire server process!

2. The asyncHandler Solution:

```
export const asyncHandler = (fn: RequestHandler) =>
  (req: Request, res: Response, next: NextFunction) => {
    Promise.resolve(fn(req, res, next)).catch(next);
  };
```

3. Result: Eliminates 500 lines of repetitive try/catch boilerplate across controllers. Any rejected promise or thrown exception is cleanly routed to our centralized errorHandler middleware.

`Code File: server/src/utils/asyncHandler.ts`

Q25. Explain your Custom ApiError Class and Centralized Error Handling Middleware.

Examiner Focus: Production-grade error handling. | Hindi Tip: Custom ApiError class me statusCode, message, aur details array hota hai. Global error handler uniform JSON response deta hai aur production me stack trace hide karta hai.

1. Custom ApiError Class (server/src/utils/ApiError.ts):
Extends JavaScript's native Error class:
- statusCode: number (400, 401, 403, 404, 409, 429, 500)
- isOperational: boolean (distinguishes known operational errors from programmer bugs)
- Factory helpers: ApiError.badRequest(), ApiError.notFound(), ApiError.unauthorized(), ApiError.forbidden().

2. Global Error Middleware (server/src/middleware/errorHandler.ts):
Catches all errors and formats a uniform JSON response:
{

```
  "success": false,
  "error": {
    "code": "NOT_FOUND",
    "message": "Shop not found",
    "details": []
  }
}
```

Security Note: Stack traces are only included in development mode (process.env.NODE_ENV !== 'production') to prevent information leakage to attackers.

`Code Files: server/src/utils/ApiError.ts & server/src/middleware/errorHandler.ts`

Q26. How are File Uploads handled and secured?

Examiner Focus: Media handling and file vulnerability defense. | Hindi Tip: Multer middleware use kiya hai. UUID file renaming se overwrite rokta hai, MIME-type whitelist sirf JPG/PNG/WebP allow karti hai, aur 5MB file cap lagaya hai.

Shopzy handles image uploads (shop storefronts, menus, profile avatars) via Multer:

1. Collision Prevention: Uploaded files are renamed using uuidv4() + original extension (e.g. 550e8400-e29b.webp). Users cannot overwrite existing files or execute path traversal attacks (../../etc/passwd).
2. Strict MIME-Type Whitelist: Only image/jpeg, image/png, and image/webp are accepted. Executables (.exe, .sh, .php, .svg with embedded XSS) are rejected immediately.
3. Size Limitations: Capped at 5MB per file and a maximum of 10 files per upload to prevent server storage denial-of-service.
4. Cleanup on Failure: If database record creation fails after file upload, a post-error cleanup hook deletes the orphaned file from disk.

`Code File: server/src/middleware/upload.ts`

SECTION 7: COMPLEX BUSINESS LOGIC & FEATURE DEEP-DIVE

Opening hours logic, asynchronous analytics recording, verification state machines, and audit trails.

Q27. How does the server determine if a shop is 'Open Now'? (Timezone & Schedule Logic)

Examiner Focus: Real-time operational status logic. | Hindi Tip: 7 days ka schedule array hota hai. Server current Indian Standard Time (IST) ka day aur time check karta hai aur compare karta hai open aur close time string se.

1. Schedule Storage: Shop model stores a 7-day schedule inside operatingHours:
   { monday: { isOpen: true, open: '09:00', close: '22:00' }, ... }

2. Real-Time Evaluation (server/src/utils/time.ts -> isOpenNow):
   - Extracts current date & time in Asia/Kolkata timezone (UTC+05:30).
   - Matches day of week (e.g. 'tuesday').
   - If schedule[day].isOpen === false -> Returns { isOpen: false, reason: 'Closed today' }.
   - Converts current time ('14:30') into integer minutes (14 * 60 + 30 = 870 mins).
   - Converts open ('09:00' -> 540) and close ('22:00' -> 1320).
   - Handles overnight shifts (e.g. open 20:00 to 02:00 next morning).
   - Evaluates: currentMinutes >= openMinutes && currentMinutes < closeMinutes.
3. This is projected as openNow: boolean on all search results and shop details.

`Code Reference: server/src/utils/time.ts & server/src/services/shop.serializer.ts`

Q28. How are Shop Analytics (Views, Calls, Directions) recorded without slowing down the user? (Fire-and-Forget)

Examiner Focus: Asynchronous event recording and non-blocking APIs. | Hindi Tip: Fire-and-forget pattern use kiya hai. Response turant return ho jata hai aur background me void recordAnalyticsEvent() asynchronously analytics update karta hai.

1. The Latency Problem: When a user visits a shop page, recording a pageview involves inserting an Analytics event document and incrementing a counter in MongoDB. If awaited synchronously, the user must wait an extra 20-30ms for the page to load.

2. The Fire-and-Forget Pattern:
   In shop.controller.ts (getShopBySlug):

```
// 1. Fetch shop data and respond to user immediately
const shop = await shopService.getShopDetail(slug);
// 2. Fire-and-forget analytics in background (notice void operator, unawaited)
void analyticsService.recordEvent({
  shopId: shop._id,
  type: 'view',
  userId: req.user?.id,
  ip: req.ip
});
return ok(res, serializeShopDetail(shop));
```

3. The HTTP response is delivered to the client instantly, while Node's event loop handles the DB write asynchronously.

`Code Reference: server/src/controllers/shop.controller.ts (Line 20 & 110)`

Q29. Explain the Shop Verification State Machine and Admin Moderation Workflow.

Examiner Focus: Business workflow and access control. | Hindi Tip: Shop creation par status 'pending' hota hai. Admin inspect karke 'verified' ya rejection reason ke saath 'rejected' mark karta hai. Search me sirf verified shops aati hain.

1. State Machine: verification.status has 4 defined enum states:
   ['pending', 'under_review', 'verified', 'rejected']

2. Workflow:
   - Step 1: Owner creates shop -> Default status is 'pending', verified: false.
   - Step 2: Public search queries filter by { 'verification.status': 'verified', isActive: true }, so unverified shops cannot appear in public search.
   - Step 3: Super Admin / Admin accesses /api/v1/admin/shops?verificationStatus=pending queue.
   - Step 4: Admin clicks 'Verify' -> Server transitions status to 'verified', attaches verifiedAt timestamp, sets verifiedBy adminId, and generates an Audit Log.
   - Step 5: If rejected, admin must provide rejectionReason, which is returned to the owner dashboard.

```
Code Reference: server/src/controllers/admin.controller.ts (verifyShop) & server/src/models/Shop.ts
```

Q30. What is the Audit Log system and why is it critical?

Examiner Focus: Compliance, traceability, and administrative governance. | Hindi Tip: Audit log me record hota hai ki kis admin ne kab, kis IP se, kya action kiya (jaise shop verify kiya, role change kiya, ya review delete kiya). Ye immutable hota hai.

1. Why It's Critical: In enterprise systems, administrators have high privileges. If an admin maliciously deletes a shop or elevates a user to super_admin, there must be an immutable paper trail.
2. Audit Model (server/src/models/AuditLog.ts):
   Stores:
   - actorId: Admin User ObjectId
   - action: 'shop.verify' | 'user.role_change' | 'shop.delete' | 'category.create'
   - targetId & targetType
   - ipAddress & userAgent
   - diff: { before, after } snapshots
   - timestamp
3. Read-Only: The /api/v1/admin/audit endpoint allows viewing but has NO delete or update endpoints, guaranteeing audit log immutability.

```
Code Reference: server/src/services/audit.service.ts & server/src/models/AuditLog.ts
```

SECTION 8: TESTING STRATEGY, DEVOPS & VIVA SUCCESS TIPS

Automated test coverage with Jest/Supertest, deployment readiness, and 5 golden presentation tips.

Q31. How did you test your backend? Explain your Jest & Supertest setup (150 Tests).

Examiner Focus: Quality assurance & testing rigor. | Hindi Tip: Jest aur Supertest se 150 automated tests likhe hain. In-memory MongoDB use kiya hai taaki testing fast ho aur real database tamper na ho.

1. Testing Stack:
- Jest: Test runner and assertion library.
- Supertest: Simulates HTTP requests against Express app without listening on a live network port.
- mongodb-memory-server: Spins up an ephemeral, in-memory MongoDB instance for tests. Tests run completely isolated from production data.

2. Test Suites Covered (150 Tests Passed):
- auth.test.ts: User registration, login, token refresh, bcrypt hashing, invalid credentials.
- shops.test.ts: CRUD, 2dsphere proximity search, category filtering, owner permission gates.
- reviews-admin.test.ts: Rating calculations, spam review moderation, review deletion.
- admin-access.test.ts: RBAC gates blocking customers/owners from accessing admin routes.
- regressions.test.ts: Redis failure fallback resilience, race condition verification.

3. Run Command: npm run test -> All 7 test suites pass in ~29 seconds.

```
Evidence: server/package.json -> 'test': 'jest --runInBand'
```

Q32. If 100,000 users visit Shopzy simultaneously, what will break first and how would you scale?

Examiner Focus: High-level systems design and horizontal scalability question. | Hindi Tip: Pehle Node.js single thread CPU saturate hoga, phir DB connections. Hum Docker containers banakar Nginx load balancer ke peeche horizontal scale karenge aur Redis cluster use karenge.

1. Bottlenecks at Scale:
- Bottleneck 1: Node.js single-threaded CPU saturation handling encryption and JSON serialization.
- Bottleneck 2: MongoDB connection pool exhaustion (50 max connections).

2. Step-by-Step Scaling Strategy:
- Step 1 (Horizontal Scaling): Containerize the server with Docker. Deploy 4+ instances behind an Nginx Reverse Proxy / AWS Application Load Balancer with round-robin traffic distribution.
- Step 2 (Database Read-Replicas): Configure MongoDB Replica Set (1 Primary for writes, 2 Secondaries for geo-queries and reads with readPreference: 'secondaryPreferred').
- Step 3 (Distributed Caching): Scale Redis into a Redis Cluster with Redis Sentinel for automatic failover.
- Step 4 (CDN for Static Assets): Serve shop cover images and frontend bundles through Cloudflare or AWS CloudFront CDN to offload 80% of bandwidth.

```
Architecture Target: AWS ECS / Kubernetes + MongoDB Atlas + Redis Cloud
```

TOP 5 GOLDEN RULES TO SCORE FULL MARKS IN YOUR VIVA TOMORROW:

1. Start with the Problem Statement: 'Sir, local shopkeepers in Tier-2/3 cities lose customers to Amazon/Blinkit because their live prices and availability are invisible online. Shopzy solves this hyperlocally.'
2. Flaunt the Tech Stack with Confidence: Don't just say 'MERN stack'. Say: 'We built a type-safe TypeScript backend using Express, MongoDB 2dsphere indexing for spherical geo-queries, Redis cache-aside layer, and dual-token JWT authentication.'
3. Show Code Files when asked: If examiner asks about security, show server/src/middleware/auth.ts. If they ask about geo-queries, show server/src/models/Shop.ts line with 2dsphere index. Showing exact code proves YOU built it.
4. Mention Test Coverage: Tell the examiner: 'Sir, we didn't just build it, we wrote 150 automated unit and integration tests using

Jest and Supertest to verify authentication, race conditions, and role gates.' (Examiners LOVE this).

5. Stay Calm on Edge Cases: If they ask something unexpected (like 'What if server loses internet?'), answer gracefully: 'That's a great edge case, sir! Currently we log an error via Winston, but in our next release we plan to queue background syncs via RabbitMQ.'